Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země

Jan Bartůšek, Petr Havel

Matematická kartografie

Konstrukce polyedrických glóbů

1. Zadání

Vybraná platonská tělesa (šestistěn, čtyřstěn a osmistěn nebo dvanáctistěn) použijte pro polyedrickou aproximaci sféry. Na plošky platonských těles znázorněte v gnomonické projekci

$$\begin{aligned} x &= R \cdot \tan(90^\circ - \check{s}) \cos d, \\ y &= R \cdot \tan(90^\circ - \check{s}) \sin d, \end{aligned}$$

geografickou síť doplněnou zákresem kontinentů. Skript pro generování sítě poledníků, rovnoběžek a znázornění kontinentů realizujte v programu MATLAB bez použití externích funkcí. Soubory se zákresem kontinentů exportujte z vhodné datové sady.

Vytvořte prostorové modely polyedrických globů v měřítku 1:100 000 000 nebo 1:50 000 000 dle možnosti Vaší tiskárny a pošlete fotografii/video Vašeho glóbu. Součástí úlohy bude příloha s rozloženými modely globů.

Zjistěte následující vlastnosti gnomonické projekce v okrajovém bodě Q jedné ze stěn Platonského tělesa:

- měřítko m_p v poledníku, měřítko m_r v rovnoběžce,
- poloosy a, b Tissotovy indikatrix,
- úhel ω' mezi obrazem poledníku a rovnoběžky,
- maximální úhlové zkreslení $\Delta\omega$,
- měřítko ploch P.

Vypočtené parametry v bodě Q použijte k zákresu Tissotovy indikatrix (doporučené měřítko 1:1 000 000), jako podklad využijte obraz geografické sítě na příslušné stěně Platonského tělesa v gnomonické projekci, volte $\Delta u = \Delta v = 10^\circ$ (formát A4, popř. odpovídající).

Výpočty proveďte pro referenční kouli s poloměrem $R = 6380$ km, hodnoty měřítek a poloos Tissotovy indikatrix uveďte s přesností na 6 desetinných míst, úhlové hodnoty s přesností na $''$. Výsledky zkontrolujte s hodnotami získanými z programu Proj.4.

Doba zpracování: 4 týdny.

Hodnocení:

Krok	**Hodnocení**
Konstrukce polyedrických globů (4- a 8- nebo 12- stěn) .	20b
Konstrukce poyedrického globu (20- stěn).	*+10b*
Digitální 3D model globu.	*+10b*
Mapa na povrchu globu.	*+5b*
Skript v Pythonu pro tvorbu 3D modelu s mapou na povrchu	*+20b*
Výpočet meridánové konvergence.	*+5b*
Kontrola zkreslení v knihovně Proj.4.	*+15b*
Prostorový model na 3D tiskárně měřítku 1:100 000 000 nebo 1:50 000 000.	*+10b*
Max celkem:	**95b**

Přehled bonusových úloh a bodování

Popis bonusové úlohy ze zadání	*Hodnocení*
Kontrukce polyedrických globů (4– a 8- stěn)	20
Konstrukce polyedrického globu (20- stěn).	+10 b.
Digitální 3D model globu.	+10 b.
Mapa na povrchu globu.	+5 b.
Skript v Pythonu pro tvorbu 3D modelu s mapou na povrchu	+20 b.
Výpočet meridánové konvergence.	+5 b.
Kontrola zkreslení v knihovně Proj.4.	+15 b.
Prostorový model na 3D tiskárně	+10 b.
Celkem	20+75 b.

2. Popis problému

Konstrukce polyedrického glóbu spočívá v aproximaci referenční sféry vybraným platonským tělesem. Základním předpokladem metody je, že rovina definovaná stěnou tělesa a středem vepsané sféry protíná tuto sféru v hlavní kružnici (ortodromě). Obrazem ortodromy v gnomonické projekci je vždy polopřímka nebo úsečka, což zaručuje, že hranice plošek se transformují do přímých linií a sféru tak lze na plošky tělesa rozvinout beze spár a překrytů.

2.1 Gnomonická projekce a transformace souřadnic

Pro zobrazení sítě a polohopisu je využita gnomonická projekce v obecné poloze. Středem promítání je střed sféry a promítací rovina je tečná ke sféře v tzv. kartografickém pólu stěny $K = [u_k, v_k]$, který se nachází v těžišti dané stěny tělesa. Zobrazovací rovnice v pravoúhlých souřadnicích mají tvar:

$$x = R \cdot \tan(90° - \tilde{s}) \cos d$$

$$y = R \cdot \tan(90° - \tilde{s}) \sin d$$

kde R je poloměr referenční koule, $\tilde{s}$ je kartografická šířka a d je kartografická délka.

Pro zamezení chyb pramenících z nejednoznačnosti goniometrických funkcí (především u obrysů kontinentů protínajících více kvadrantů) je nezbytné aplikovat kvadrantově korektní převod ze standardních geografických souřadnic $[u, v]$ do kartografických souřadnic $[\tilde{s}, d]$:

$$\sin \tilde{s} = \sin u \sin u_k + \cos u \cos u_k \cos(v_k - v)$$

$$d = \operatorname{atan2}(\sin(v_k - v)\cos u, \cos u \sin u_k \cos(v_k - v) - \sin u \cos u_k)$$

2.2 Geometrie pravidelného šestistěnu (krychle)

Stěny krychle tvoří šest stejných čtverců. Poloměr sféry vepsané do krychle s délkou hrany a odpovídá přesně polovině hrany, tedy: $r = a/2$

Pro určení souřadnic vrcholů hledáme zeměpisnou šířku u, kterou lze odvodit z poměru hrany a stěnové úhlopříčky. Úhel představující tuto šířku splňuje podmínku: $\tan u = \frac{1}{\sqrt{2}}$ Což odpovídá hodnotě $u \approx 35.2644°$. Čtyři vrcholy leží na severní polokouli (E, F, G, H) a čtyři symetricky na jižní (A, B, C, D). Kartografické póly jsou voleny ve středech čtvercových stěn. Čtyři póly leží na rovníku s rozestupem 90° (např. pól K s délkou 45°) a zbylé dva představují severní a jižní pól.

2.3 Geometrie pravidelného čtyřstěnu

Stěny pravidelného čtyřstěnu tvoří čtyři rovnostranné trojúhelníky. Poloměr vepsané sféry r tělesu o hraně a je dán vztahem $r = \frac{\sqrt{3}}{6}a$. Kartografické póly (těžiště plošek) a vrcholy vykazují specifickou symetrii. Zeměpisná šířka u vrcholů ležících na jižní polokouli (a odpovídajících těžišť na severní) splňuje podmínku:

$$\sin u = \frac{1}{3} \Rightarrow u \approx 19.4712°$$

2.4 Geometrie pravidelného osmistěnu

Pravidelný osmistěn je tvořen osmi rovnostrannými trojúhelníky. Poloměr vepsané sféry r je definován jako $r = \frac{a}{\sqrt{6}}$. Vrcholy tělesa při standardní orientaci tvoří severní a jižní pól a čtyři body na rovníku s délkami odstupňovanými po 90°. Zeměpisná šířka kartografických pólů (těžišť stěn) je určena vztahem:

$$\sin u = \frac{\sqrt{3}}{3} \Rightarrow u \approx 35.2644°$$

3. Výpočty

3.1 Vstupní parametry

Veškeré výpočty charakteristik zkreslení jsou exaktně odvozeny a vztaženy k referenční kouli o poloměru $R = 6380$ km. Výpočet je demonstrován pro okrajový bod Q (vrchol tělesa) definované stěny, kde gnomonická projekce dosahuje maximálního zkreslení.

3.2 Analýza zkreslení pro šestistěn (krychli)

Pro výpočet zvolíme okrajový bod Q představující vrchol E ($u_E = 35.264390°, v_E = 0°$) vzhledem ke kartografickému pólu stěny K na rovníku ($u_K = 0°, v_K = 45°$).

Nejprve určíme kartografickou šířku $\tilde{s}$. Vzhledem k tomu, že $\sin(u_K) = 0$ a $\cos(u_K) = 1$, rovnice se výrazně zjednoduší:

$$\sin \tilde{s} = \sin(35.264390°) \cdot 0 + \cos(35.264390°) \cdot 1 \cdot \cos(45° - 0°)$$

Geometricky platí, že $\cos(35.264390°) = \frac{\sqrt{2}}{\sqrt{3}}$ a $\cos(45°) = \frac{1}{\sqrt{2}}$.

Po dosazení získáme:

$$\sin \tilde{s} = \frac{\sqrt{2}}{\sqrt{3}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{\sqrt{3}} \approx 0.577350$$

Tato hodnota je naprosto identická s výsledkem u pravidelného osmistěnu (vlivem tzv. duality těchto dvou platonských těles). Sférická vzdálenost k vrcholu je i u krychle přibližně z $\approx 54.735610°$. Extrémní hodnoty zkreslení jsou proto totožné:

$$a = \frac{1}{\sin^2(\tilde{s})} = 3.000000$$

$$b = \frac{1}{\sin(\tilde{s})} = \sqrt{3} \approx 1.732051$$

$$P = a \cdot b = 3.000000 \cdot 1.732051 = 5.196152$$

$$\Delta\omega = 31°05'04''$$

3.3 Analýza zkreslení pro čtyřstěn

Pro výpočet zvolíme vrchol $A(u_A = -19.471221°, v_A = 0°)$ vzhledem ke kartografickému pólu K $(u_K = 19.471221°, v_K = 60°)$.

Kartografická šířka $\tilde{s}$ bodu Q je:

$$\sin \tilde{s} = \left(-\frac{1}{3}\right)\left(\frac{1}{3}\right) + \left(\frac{\sqrt{8}}{3}\right)\left(\frac{\sqrt{8}}{3}\right)\cos(60°) = \frac{1}{3}$$

Sférická vzdálenost od pólu $z = 90° - \tilde{s} \approx 70.528779°$. Vzhledem k tomu, že gnomonická projekce je azimutální, hlavní směry zkreslení leží v radiálním (a) a transverzálním (b) směru:

$$a = \frac{1}{\sin^2 \tilde{s}} = \frac{1}{(1/3)^2} = 9.000000$$

$$b = \frac{1}{\sin \tilde{s}} = \frac{1}{1/3} = 3.000000$$

Plošné zkreslení $P = a \cdot b = 27.000000$.

Maximální úhlové zkreslení: $\sin\left(\frac{\Delta\omega}{2}\right) = \frac{9-3}{9+3} = 0.5 \Rightarrow \Delta\omega = 60°00'00''$.

3.4 Analýza zkreslení pro osmistěn

Pro osmistěn zvolíme stěnu definovanou na severní polokouli pólem I $(u_I = 35.264390°, v_I = 45°)$ a okrajovým bodem Q na severním pólu $(u_Q = 90°)$.

Kartografická šířka $\tilde{s}$ bodu Q je:

$$\sin \tilde{s} = \sin(90°)\sin(35.264390°) + 0 = \frac{\sqrt{3}}{3} \approx 0.577350$$

Sférická vzdálenost z $\approx 54.735610°$.

Extrémní hodnoty zkreslení:

$$a = \frac{1}{\sin^2 \tilde{s}} = \frac{1}{\left(\sqrt{3}/3\right)^2} = 3.000000$$

$$b = \frac{1}{\sin \tilde{s}} = \frac{1}{\sqrt{3}/3} = \sqrt{3} \approx 1.732051$$

Plošné zkreslení $P = a \cdot b = 3.000000 \cdot 1.732051 = 5.196152$.

Maximální úhlové zkreslení: $\sin\left(\frac{\Delta\omega}{2}\right) = \frac{3-\sqrt{3}}{3+\sqrt{3}} \approx 0.267949 \Rightarrow \Delta\omega = 31°05'04''$.

3.5 Analýza zkreslení pro dvacetistěn (ikozaedr)

Pravidelný dvacetistěn představuje nejvyšší stupeň aproximace sféry mezi platonskými tělesy. Vzdálenost od těžiště stěny k jejímu vrcholu je zde nejmenší, což přímo generuje nejnižší hodnoty zkreslení.

Pro výpočet sférické vzdálenosti z okrajového bodu Q od pólu zobrazení využijeme vztah vycházející z poloměru vepsané a opsané sféry ikozaedru: $\cos z = \frac{\sqrt{3}}{3}\cot\left(\frac{\pi}{5}\right) \approx 0.794654$

Tomu odpovídá sférická vzdálenost $z \approx 37.377368°$ (respektive kartografická šířka $\{s\} \approx 52.622632°$).

Charakteristiky zkreslení v okrajovém bodě Q:

Poloosy Tissotovy indikatrix $b = \frac{1}{\sin(\tilde{s})} = \frac{1}{0.794654} \approx 1.258409 a = \frac{1}{\sin^2(\tilde{s})} = \frac{1}{0.794654^2} \approx 1.583592$

Plošné zkreslení $P = a \cdot b = 1.583592 \cdot 1.258409 \approx 1.992806$

Maximální úhlové zkreslení $\sin\left(\frac{\Delta\omega}{2}\right) = \frac{a-b}{a+b} = \frac{1.583592-1.258409}{1.583592+1.258409} \approx 0.114420 \frac{\Delta\omega}{2} \approx 6.568138° \Rightarrow \Delta\omega = 13°08'11''$

4. Skripty a programové zpracování

V této části je detailně popsána programová realizace výpočtů a vykreslování polyedrických globů v prostředí MATLAB. Celé řešení je postaveno na modulární architektuře vlastních funkcí, které řeší specifické kartografické problémy (transformace, singularity zobrazení, topologická integrita linií) bez závislosti na externích mapových knihovnách. Níže jsou postupně rozebrána algoritmická specifika a geometrické definice pro jednotlivá modelovaná tělesa.

4.1 Pravidelný šestistěn (krychle)

Implementace pro pravidelný šestistěn je řízena hlavním skriptem u2_6.m a sadou podpůrných funkcí zakončených sufixem _6.m.

1. Definice geometrie tělesa (u2_6.m)

Klíčovým parametrem pro ustanovení krychle v prostoru je zeměpisná šířka u, která definuje polohu vrcholů. Ta je z prostorové geometrie exaktně vypočtena jako $u = \operatorname{atan}\left(1/\sqrt{2}\right)$.

```
% Geometry of the cube-octahedron face orientation
u = atan(1 / sqrt(2));   % approx. 35.2644°

% Face corner vertices in geographic coordinates
A = [-u,   0*pi/180];
B = [-u,  90*pi/180];
C = [-u, 180*pi/180];
D = [-u, 270*pi/180];

E = [ u,   0*pi/180];
F = [ u,  90*pi/180];
G = [ u, 180*pi/180];
H = [ u, 270*pi/180];
```

Tyto souřadnice, společně s kartografickými póly stěn (např. $K = [0{,}45 * pi/180]$), jsou uloženy do struktury faces, což umožňuje vykreslovat celý glóbus automatizovaně v cyklu.

2. Transformace do obecné polohy (uvTosd_6.m)

Tato funkce řeší převod ze zeměpisných souřadnic [u, v] do kartografických [s, d] vzhledem k pólu zobrazení [uk, vk]. Implementace striktně využívá kvadrantově korektní výpočet délky, aby nedocházelo k chybám při přechodu přes 180. poledník. Zároveň je zde ošetřena numerická stabilita funkce arkus sinus:

```
% Convert geographic coordinates [u, v] to rotated spherical coordinates [s, d]
% relative to the cartographic face center [uk, vk].

 dv = vk - v;

 arg = sin(u).*sin(uk) + cos(u).*cos(uk).*cos(dv);
 arg = max(min(arg, 1), -1);
 s = asin(arg);

 num = sin(dv).*cos(u);
 den = cos(u).*sin(uk).*cos(dv) - sin(u).*cos(uk);

 d = -atan2(num, den);
```

3. Gnomonická projekce (gnom_6.m)

Vlastní průmět kartografických souřadnic do tečné roviny stěny využívá klasické zobrazovací rovnice azimutální projekce: $\rho = R \cdot \tan(\pi/2 - \tilde{s})$

```
% Project spherical coordinates to the gnomonic plane.

rho = R * tan(pi/2 - s);
x = rho .* cos(d);
y = rho .* sin(d);
```

4. Generování sítě a ošetření singularit (graticule_6.m)

Protože gnomonická projekce promítá body vzdálené 90° od pólu do nekonečna, zavádí skript bezpečnostní limit. Odstraní všechny body, jejichž kartografická šířka klesne pod 5°.

```
% Generate projected meridians and parallels and clip them to the face boundary.

sMin = 5 * pi/180;   % protect against the gnomonic projection singularity

% Poledniky
XM = [];
YM = [];

for v = vmin:Dv:vmax
    um = umin:du:umax;
    vm = ones(size(um)) * v;

    [sm, dm] = uvTosd_6(um, vm, uk, vk);

    valid = sm > sMin;

    xm = NaN(size(sm));
    ym = NaN(size(sm));

    if any(valid)
        [xv, yv] = gnom_6(R, sm(valid), dm(valid));
        xm(valid) = xv;
        ym(valid) = yv;
    end
```

5. Ořezávání a topologická kontrola (drawContinent_6.m a clipLineToPolygon_6.m)

Po projekci je nutné kontinenty oříznout do přesného tvaru stěny (čtverce). Skript nejprve aplikuje polygonový ořez (z boundary_6.m) a následně kontroluje euklidovskou vzdálenost lomových bodů. Pokud je vzdálenost nepřiměřeně velká (čára protíná horizont projekce), vloží hodnotu NaN a tím linii přeruší.

```
% Clip the continent outline to the actual triangular face boundary
[XC, YC] = clipLineToPolygon_6(XC, YC, polyX, polyY);

% Split lines at large jumps to avoid drawing invalid connection segments
dx = max(polyX) - min(polyX);
dy = max(polyY) - min(polyY);
jumpThreshold = 0.75 * max(dx, dy);

for i = 1:length(XC)-1
    if ~isnan(XC(i)) && ~isnan(XC(i+1)) && ~isnan(YC(i)) && ~isnan(YC(i+1))
        if hypot(XC(i+1)-XC(i), YC(i+1)-YC(i)) > jumpThreshold
            XC(i+1) = NaN;
            YC(i+1) = NaN;
        end
    end
end
```

6. Výpočet meridiánové konvergence (meridianConvergence.m)

V rámci rozšiřujících úloh je vypočtena i meridiánová konvergence pomocí numerické derivace. Skript počítá diference promítnutých souřadnic poledníku při mikroskopickém posunu v šířce (h = 1e-6), což poskytuje analyticky přesný směrový úhel α:

```
% Estimate the meridian convergence at a geographic point using finite difference

h = 1e-6;

u1 = uQ - h;
u2 = uQ + h;

v1 = vQ;
v2 = vQ;

[s1, d1] = uvTosd_fun(u1, v1, uk, vk);
[s2, d2] = uvTosd_fun(u2, v2, uk, vk);

[x1, y1] = gnom_fun(R, s1, d1);
[x2, y2] = gnom_fun(R, s2, d2);

dx = x2 - x1;
dy = y2 - y1;

alpha = atan2(dy, dx);
gamma = pi/2 - alpha;
```

4.2 Pravidelný čtyřstěn (tetraedr)

Implementace pro pravidelný čtyřstěn je řízena hlavním skriptem u2_4.m. Přestože sdílí základní principy gnomonické projekce se šestistěnem, jeho prostorová geometrie vyžaduje zcela odlišnou definici pólů a ořezových polygonů (rovnostranných trojúhelníků). V rámci těchto skriptů je navíc implementováno elegantní řešení pro výpočet meridiánové konvergence pomocí numerické derivace.

1. Definice geometrie tělesa (u2_4.m)

Základním krokem je určení zeměpisné šířky u pro vrcholy na jižní polokouli. Z prostorových vlastností tělesa je v kódu analyticky určena jako $u = \arcsin(1/3)$. Rozestupy mezi jednotlivými vrcholy v délce činí přesně 120°.

```
% GEOMETRY

u = asin(1/3);

A = [-u, 0];
B = [-u, 120*pi/180];
C = [-u, 240*pi/180];
D = [ pi/2, 0];

K = [ u,  60*pi/180];
L = [ u, 180*pi/180];
M = [ u, 300*pi/180];
N = [-pi/2, 0];
```

Tyto vrcholy a kartografické póly plošek (K, L, M, N) tvoří definiční rámec uložený do struktury faces.

2. Transformace do obecné polohy (uvTosd_4.m)

Funkce spolehlivě převádí geografické souřadnice na kartografické. Důraz je zde opět kladen na ošetření argumentu arkus sinu a kvadrantovou korektnost délky:

```
% Converts geographic coordinates [u,v] to cartographic coordinates

dv = vk - v;

% kartograficka sirka
arg = sin(u).*sin(uk) + cos(u).*cos(uk).*cos(dv);
arg = max(min(arg, 1), -1);
s = asin(arg);

% kartograficka delka - kvadrantove korektne
num = sin(dv).*cos(u);
den = cos(u).*sin(uk).*cos(dv) - sin(u).*cos(uk);

d = -atan2(num, den);
```

3. Gnomonická projekce (gnom_4.m)

Vlastní průmět využívá čistě vektorizovaný zápis rovnic pro maximální výpočetní efektivitu:

```
% Gnomonic projection: transforms cartographic coordinates to planar coordinates

rho = R * tan(pi/2 - s);
x = rho .* cos(d);
y = rho .* sin(d);
```

4. Generování sítě a ošetření singularit (graticule_4.m)

Funkce pro generování sítě obsahuje ochranu před singularitou gnomonické projekce (odstranění bodů blízko horizontu, tedy 90° od pólu zobrazení):

```
% Creates geographic graticule (meridians and parallels) in gnomonic projection

sMin = 5 * pi/180;   % Protection from gnomonic projection singularity
```

Vygenerované linie jsou následně okamžitě ořezávány funkcí clipLineToPolygon_4.m podle trojúhelníkové hranice stěny.

5. Topologická kontrola při vykreslování kontinentů (drawContinent_4.m)

Při zobrazení reálných dat z textových souborů je nutné zabránit nežádoucímu propojení lomových bodů ležících na opačných stranách zobrazené polokoule. Skript aplikuje dynamický práh, který se vypočítá z rozměrů hraničního trojúhelníku stěny:

```
% Clipping to the actual face triangle
[XC, YC] = clipLineToPolygon_4(XC, YC, polyX, polyY);

% Safety split for large jumps
dx = max(polyX) - min(polyX);
dy = max(polyY) - min(polyY);
jumpThreshold = 0.75 * max(dx, dy);

for i = 1:length(XC)-1
    if ~isnan(XC(i)) && ~isnan(XC(i+1)) && ~isnan(YC(i)) && ~isnan(YC(i+1))
        if hypot(XC(i+1)-XC(i), YC(i+1)-YC(i)) > jumpThreshold
            XC(i+1) = NaN;
            YC(i+1) = NaN;
        end
    end
end
```

6. Výpočet meridiánové konvergence (meridianConvergence.m) – Rozšiřující úloha

Velmi cenným přínosem tohoto programového bloku je vytvoření univerzálního nástroje pro výpočet meridiánové konvergence. Namísto složitého analytického derivování zobrazovacích rovnic skript využívá metodu numerické derivace s krokem h = 1e-6. Tento postup spolehlivě získá směrový úhel α (potažmo γ):

```
% Calculates meridian convergence angle using numerical differentiation

h = 1e-6;

u1 = uQ - h;
u2 = uQ + h;

v1 = vQ;
v2 = vQ;

[s1, d1] = uvTosd_fun(u1, v1, uk, vk);
[s2, d2] = uvTosd_fun(u2, v2, uk, vk);

[x1, y1] = gnom_fun(R, s1, d1);
[x2, y2] = gnom_fun(R, s2, d2);

dx = x2 - x1;
dy = y2 - y1;

alpha = atan2(dy, dx);
gamma = pi/2 - alpha;
```

4.3 Pravidelný osmistěn (oktaedr)

Zpracování pravidelného osmistěnu (skripty *_8.m) plně těží z vytvořené modulární architektury. Algoritmus je shodný s předchozími tělesy, avšak operuje s osmi plošnými elementy (rovnostrannými trojúhelníky). Specifikem osmistěnu je orientace jeho sítě, kdy vrcholy tělesa leží přesně na severním a jižním pólu a další čtyři symetricky na rovníku.

1. Definice geometrie tělesa (u2_8.m)

Zeměpisná šířka těžišť stěn (kartografických pólů) vychází z geometrického odvození poměru mezi poloměrem vepsané sféry a tělesovou úhlopříčkou. Konstanta u je definována analyticky jako $u = \operatorname{asin}(\sqrt{3}/3)$.

```
% Geometry parameters

u = asin(sqrt(3)/3);   % approximately 35.2644°

% Corner vertices of the octahedron faces
A = [0, 0];
B = [0,  90*pi/180];
C = [0, 180*pi/180];
D = [0, 270*pi/180];
E = [ pi/2, 0];
F = [-pi/2, 0];
```

Tyto vrcholy tvoří ohraničení pro čtyři stěny na severní polokouli (sdružené kolem pólu E) a čtyři stěny na jižní polokouli (kolem pólu F).

2. Transformace do obecné polohy (uvTosd_8.m)

Stejně jako u ostatních těles je implementován kvadrantově korektní výpočet kartografické délky:

```
% uvTosd_8 transforms geographic coordinates [u, v] to the face coordinate system

dv = vk - v;

% Compute the face latitude (s).
arg = sin(u).*sin(uk) + cos(u).*cos(uk).*cos(dv);
arg = max(min(arg, 1), -1);
s = asin(arg);

% Compute the face longitude (d) with correct quadrant handling.
num = sin(dv).*cos(u);
den = cos(u).*sin(uk).*cos(dv) - sin(u).*cos(uk);

d = -atan2(num, den);
```

3. Gnomonická projekce (gnom_8.m)

Vektorizovaný zápis rovnic pro efektivní výpočet pravoúhlých souřadnic na tečné rovině stěny:

```
% gnom_8 performs the gnomonic projection of face coordinates.

rho = R * tan(pi/2 - s);
x = rho .* cos(d);
y = rho .* sin(d);
```

4. Generování sítě a ošetření singularit (graticule_8.m)

Rovněž v případě osmistěnu je zachována pojistka proti nekonečnu gnomonické projekce (odstranění bodů v blízkosti horizontu):

```
% graticule_8 generates projected meridian and parallel lines clipped to a face.

sMin = 5 * pi/180;   % safeguard against the gnomonic projection singularity
```

5. Topologická kontrola a polygonální ořez (drawContinent_8.m, clipLineToPolygon_8.m)

Linie kontinentů jsou ořezány podle přesných hranic rovnostranných trojúhelníků osmistěnu. Funkce zároveň rozpojuje dlouhé vektory vzniklé nechtěným přechodem přes polokouli:

```
% Clip continent lines to the actual face polygon.
[XC, YC] = clipLineToPolygon_8(XC, YC, polyX, polyY);

% Split lines at large jumps to avoid drawing spurious segments.
dx = max(polyX) - min(polyX);
dy = max(polyY) - min(polyY);
jumpThreshold = 0.75 * max(dx, dy);

for i = 1:length(XC)-1
    if ~isnan(XC(i)) && ~isnan(XC(i+1)) && ~isnan(YC(i)) && ~isnan(YC(i+1))
        if hypot(XC(i+1)-XC(i), YC(i+1)-YC(i)) > jumpThreshold
            XC(i+1) = NaN;
            YC(i+1) = NaN;
        end
    end
end
```

6. Výpočet meridiánové konvergence (testConvergence_8.m)

Univerzální funkce využívající numerickou derivaci (meridianConvergence.m) z předchozích kroků je zde aplikována přímo na konkrétní souřadnice plošky osmistěnu:

```
% Compute the meridian convergence at the sample point.
gamma = meridianConvergence(Q(1), Q(2), I(1), I(2), R, @uvTosd_8, @gnom_8);
gamma_deg = gamma * 180/pi;
```

4.4 Pravidelný dvacetistěn (ikozaedr) – Rozšiřující úloha

Dvacetistěn aproximuje sféru pomocí 20 rovnostranných trojúhelníků, což z něj činí platonské těleso s nejmenším maximálním zkreslením v okrajových oblastech. Vzhledem ke geometrické složitosti (12 vrcholů, 20 stěn) byl zcela opuštěn koncept manuálního zadávání zeměpisných souřadnic. Algoritmus těleso dynamicky generuje ve 3D kartézském prostoru a následně jej transformuje.

1. Analytická konstrukce tělesa ve 3D (buildIcosahedron_20.m)

Funkce generuje vrcholy dvacetistěnu na základě konstanty zlatého řezu $\phi = \frac{1+\sqrt{5}}{2}$. Získané prostorové souřadnice jsou následně normalizovány na jednotkovou sféru.

```
% Standard regular icosahedron in 3D
phi = (1 + sqrt(5)) / 2;

V = [
    -1,  phi,  0;
     1,  phi,  0;
    -1, -phi,  0;
     1, -phi,  0;
     0, -1,  phi;
     0,  1,  phi;
     0, -1, -phi;
     0,  1, -phi;
     phi,  0, -1;
     phi,  0,  1;
    -phi,  0, -1;
    -phi,  0,  1
];

% Normalize vertices to unit sphere
for i = 1:size(V,1)
    V(i,:) = V(i,:) / norm(V(i,:));
end
```

2. Převod 3D souřadnic na geografické (cartToGeo_20.m)

Stěžejním můstkem mezi 3D geometrií a kartografií je tato funkce, která přepočítává kartézské souřadnice [x, y, z] na geografické [u, v] (šířku a délku):

```
% Converts Cartesian coordinates to geographic coordinates (latitude, longitude)
r = sqrt(x.^2 + y.^2 + z.^2);

u = asin(z ./ r);
v = atan2(y, x);
```

Tímto způsobem se automaticky dopočítají nejen souřadnice všech 12 vrcholů, ale z průměru vrcholů (těžiště ve 3D) i kartografické póly všech 20 stěn [uk, vk].

3. Transformace do obecné polohy (uvTosd_20.m)

Tak jako u předchozích těles, vygenerované geografické souřadnice jsou následně transformovány vzhledem k exaktně vypočteným pólům stěn s ochranou argumentu arkus sinu:

```
% Converts geographic coordinates [u, v] to general position

dv = vk - v;

arg = sin(u).*sin(uk) + cos(u).*cos(uk).*cos(dv);
arg = max(min(arg, 1), -1);
s = asin(arg);
```

4. Gnomonická projekce (gnom_20.m)

Aplikace standardní zobrazovací rovnice z předchozích kroků, plně vektorizovaná pro rychlý běh celého cyklu přes 20 stěn:

```
% Gnomonic projection: converts spherical distance and azimuth
rho = R * tan(pi/2 - s);
x = rho .* cos(d);
y = rho .* sin(d);
```

5. Ošetření singularit a topologie kontinentu (drawContinent_20.m)

Vzhledem k většímu počtu (a tím pádem i menšímu plošnému rozsahu) stěn je topologická kontrola pomocí dynamického prahu (jumpThreshold) ještě kritičtější pro zajištění toho, aby se kontinentální linie netáhly přes celou konstrukci, pakliže překročí hranu ikozaedru.

```
[XC, YC] = clipLineToPolygon_20(XC, YC, polyX, polyY);

dx = max(polyX) - min(polyX);
dy = max(polyY) - min(polyY);
jumpThreshold = 0.75 * max(dx, dy);

for i = 1:length(XC)-1
    if ~isnan(XC(i)) && ~isnan(XC(i+1)) && ~isnan(YC(i)) && ~isnan(YC(i+1))
        if hypot(XC(i+1)-XC(i), YC(i+1)-YC(i)) > jumpThreshold
            XC(i+1) = NaN;
            YC(i+1) = NaN;
        end
    end
end
```

4.5 Nezávislá kontrola zkreslení (Knihovna PROJ)

V rámci rozšiřujících úloh byla provedena nezávislá kontrola vypočtených charakteristik zkreslení pomocí open-source transformační knihovny PROJ (dříve PROJ.4). Cílem bylo ověřit lokální deformace gnomonické projekce metodou numerické diferenciace nezávisle na vlastním kódu v MATLABu.

1. Metodika a parametry transformace Pro ověření byla zvolena gnomonická projekce vztažená ke stěně pravidelného osmistěnu se středem (kartografickým pólem) $K = [35.2644°, 45°]$. Jako testovací bod pro analýzu lokálního zkreslení byl vybrán bod $Q = [10°, 10°]$.

Transformace byla realizována pomocí BASH skriptu volajícího nástroj proj z příkazové řádky. Parametry definující zobrazení a referenční kouli (R = 6380 km) byly zadány standardním inicializačním řetězcem: +proj=gnom +lat_0=35.2644 +lon_0=45 +R=6380000

Pro určení diferenciálních změn délek obrazů ve směru poledníku a rovnoběžky byl zvolen velmi malý krok $h = 0.0001°$.

2. Zdrojový kód kontrolního skriptu (BASH) Skript provádí promítnutí základního bodu a dvou bodů s mikroskopickým posunem ve směru zeměpisné šířky (poledník) a délky (rovnoběžka). Následně pomocí nástroje bc vypočítá euklidovskou vzdálenost těchto bodů v projekční rovině:

```bash
#!/bin/bash
# PROJ gnomonic projection definition
P="+proj=gnom +lat_0=35.2644 +lon_0=45 +R=6380000"
# step size in degrees
h=0.0001
# base geographic point (lon lat)
Q="10 10"
# offset points for northward and eastward moves
Q_u="10 10.0001"
Q_v="10.0001 10"

# project the base and offset points
read x0 y0 <<< $(echo $Q   | proj $P)
read xu yu <<< $(echo $Q_u | proj $P)
read xv yv <<< $(echo $Q_v | proj $P)

# compute projected coordinate differences
du_x=$(echo "$xu - $x0" | bc -l)
du_y=$(echo "$yu - $y0" | bc -l)
dv_x=$(echo "$xv - $x0" | bc -l)
dv_y=$(echo "$yv - $y0" | bc -l)

# compute vector lengths in projected space
mp=$(echo "sqrt($du_x^2 + $du_y^2)" | bc -l)
mr=$(echo "sqrt($dv_x^2 + $dv_y^2)" | bc -l)

echo "mp (meridian): $mp"
echo "mr (parallel): $mr"
```

3. Analýza výsledků a určení měřítek

Z výstupu knihovny PROJ byly získány surové délky promítnutých diferenciálních úseků v rovině: mp_{raw} = 17.084074 m

mr_{raw} = 16.975541 m

Aby bylo možné určit lokální délková měřítka, bylo nutné vypočítat skutečné délky těchto úseků na referenční sféře. Krok h byl nejprve převeden na radiány: $h_{rad} = h \cdot \left(\frac{\pi}{180}\right)$

Skutečná délka oblouku poledníku (ds_{mer}) a rovnoběžky (ds_{par}) v testovacím bodě Q:

$$ds_{mer} = R \cdot h_{rad} \approx 11.135201 \text{ m} ds_{par} = R \cdot \cos(10°) \cdot h_{rad} \approx 10.966445 \text{ m}$$

Skutečná lokální měřítka v hlavních směrech geografické sítě se následně exaktně určí jako poměr délky obrazu a délky na sféře: $m_p = \frac{mp_{raw}}{ds_{mer}} \approx 1.534658 m_r = \frac{mr_{raw}}{ds_{par}} \approx 1.547975$

Z těchto hodnot je patrné, že v bodě Q dochází k výraznému zvětšení délek v obou směrech sféry. Diference mezi hodnotami prokazuje, že zkreslení gnomonické projekce není v obecném bodě izotropní. Plošné měřítko v daném bodě lze přibližně vyjádřit součinem: $P = m_p \cdot m_r \approx 2.375$

6. Generování sítě a ošetření singularit (graticule_20.m)

Funkce iteruje přes poledníky a rovnoběžky v zadaných rozsazích. Obsahuje pojistku sMin, která zabraňuje výpočtu souřadnic pro body ležící na horizontu projekce (90° od pólu), které by v mapě směřovaly do nekonečna.

```
function [XM, YM, XP, YP] = graticule_20(umin, umax, vmin, vmax, Du, Dv, du, dv,
% Generates graticule lines (meridians and parallels) for a map face
sMin = 5 * pi/180;   % Protection against singularity in gnomonic projection

% Meridians
XM = [];
YM = [];

for v = vmin:Dv:vmax
    um = umin:du:umax;
    vm = ones(size(um)) * v;

    [sm, dm] = uvTosd_20(um, vm, uk, vk);

    valid = sm > sMin;

    xm = NaN(size(sm));
    ym = NaN(size(sm));

    if any(valid)
        [xv, yv] = gnom_20(R, sm(valid), dm(valid));
        xm(valid) = xv;
        ym(valid) = yv;
    end

    [xm, ym] = clipLineToPolygon_20(xm, ym, polyX, polyY);

    XM = [XM; xm];
    YM = [YM; ym];
end

% Parallels
XP = [];
YP = [];

for u = umin:Du:umax
    vp = vmin:dv:vmax;
    up = ones(size(vp)) * u;

    [sp, dp] = uvTosd_20(up, vp, uk, vk);

    valid = sp > sMin;

    xp = NaN(size(sp));
    yp = NaN(size(sp));

    if any(valid)
        [xv, yv] = gnom_20(R, sp(valid), dp(valid));
        xp(valid) = xv;
        yp(valid) = yv;
    end

    [xp, yp] = clipLineToPolygon_20(xp, yp, polyX, polyY);

    XP = [XP; xp];
    YP = [YP; yp];
end

end
```

4.6 Prostorová 3D vizualizace a export modelu (Osmistěn)

Nad rámec tvorby 2D rozložených sítí byl v rámci rozšiřujících úloh vyvinut aparát pro plnohodnotnou 3D vizualizaci polyedrického glóbu v kartézském prostoru a jeho následný export pro účely 3D tisku a počítačové grafiky. Řešení je demonstrováno na pravidelném osmistěnu.

1. 3D vizualizace v prostředí MATLAB (u2_3D_8.m)

Tento skript sestavuje prostorový model sféry aproximované osmistěnem. Základní geografické vrcholy jsou nejprve převedeny na 3D kartézské souřadnice ležící na jednotkové kouli pomocí sférické transformace (geoToCart_8.m). Samotné těleso je vykresleno pomocí funkce patch, které je definováno maticí vrcholů V a maticí topologie stěn FACES.

Nejsložitějším matematickým problémem této části je zpětné namapování 2D vektorových kontinentů (z gnomonické projekce) na 3D stěny tělesa. Toto je řešeno ve skriptu drawFaceContinents3D_8.m za využití barycentrických souřadnic (map2DTo3DTriangle_8.m). Algoritmus určí polohu bodu uvnitř 2D polygonu pomocí vah l_1, l_2, l_3 a ty samé váhy následně aplikuje na vrcholy 3D trojúhelníku:

```
function P3 = map2DTo3DTriangle_8(x, y, polyX, polyY, tri3D)
% Map a 2D triangle point to corresponding 3D triangle using barycentric coords

x1 = polyX(1); y1 = polyY(1);
x2 = polyX(2); y2 = polyY(2);
x3 = polyX(3); y3 = polyY(3);

% Denominator for barycentric coordinate computation
den = (y2 - y3)*(x1 - x3) + (x3 - x2)*(y1 - y3);

% Barycentric weights
l1 = ((y2 - y3)*(x - x3) + (x3 - x2)*(y - y3)) / den;
l2 = ((y3 - y1)*(x - x3) + (x1 - x3)*(y - y3)) / den;
l3 = 1 - l1 - l2;

% Interpolate 3D position from barycentric weights
P3 = l1 * tri3D(1,:) + l2 * tri3D(2,:) + l3 * tri3D(3,:);
end
```

Aby se při vizualizaci zamezilo tzv. z-fightingu (prolínání čar s plochou stěny), je vypočtena normála každé stěny a mapové linie jsou mikroskopicky vysunuty nad povrch modelu (P3 = P3 + offset * n;).

2. Export modelu pro 3D tisk (u2_OBJ_8.m) Zatímco vizualizační skript linie pouze vykresluje, exportní skript (collectFaceLines3D_8.m) je shromažďuje do strukturovaného pole buněk (lineSet). Celý model je následně zvětšen do požadovaného měřítka (např. scale = 50) pro účely 3D tisku či renderování.

Samotný zápis zajišťuje funkce exportOBJ_8.m, která transformuje struktury MATLABu do standardizovaného textového formátu **Wavefront OBJ**. Skript striktně dodržuje specifikaci formátu:

- Vrcholy geometrie jsou definovány prefixem v.
- Plochy stěn jsou definovány prefixem f (face).
- Vektorové čáry (kontinenty a mřížka) jsou zapisovány prefixem l (line), přičemž indexace vrcholů čar kontinuálně navazuje na vrcholy tělesa pomocí dynamického posunu vertexOffset.

```
% Polylines: append vertices and write line elements
vertexOffset = size(V,1); % base index for subsequent appended vertices

if ~isempty(lineSet)
    for k = 1:numel(lineSet)
        P = lineSet{k};
        if isempty(P)
            continue;
        end

        % named object/group for each line
        fprintf(fid, '\no line_%04d\n', k);

        % vertices for this polyline
        for i = 1:size(P,1)
            fprintf(fid, 'v %.8f %.8f %.8f\n', P(i,1), P(i,2), P(i,3));
        end

        % Write line element referencing the newly added vertices
        fprintf(fid, 'l');
        for i = 1:size(P,1)
            fprintf(fid, ' %d', vertexOffset + i);
        end
        fprintf(fid, '\n');

        % Update offset for next polyline
        vertexOffset = vertexOffset + size(P,1);
    end
end
```

Tímto přístupem vzniká komplexní 3D model, který v sobě nese jak hmotu polyedru (plochy), tak i exaktní prostorovou kresbu linií, kterou lze ve 3D tiskových programech.

4.7 Alternativní implementace 3D vizualizace v jazyce Python

Jako doplněk k hlavnímu řešení v prostředí MATLAB byl vytvořen ekvivalentní skript v jazyce Python. Tento přístup demonstruje přenositelnost navržených matematických algoritmů do populárního open-source prostředí. Skript využívá knihovnu numpy pro vektorizované operace a matplotlib.pyplot (včetně Poly3DCollection) pro samotné 3D vykreslování stěn osmistěnu.

1. Topologický ořez pomocí objektu Path (clipLine) Na rozdíl od MATLABu, který disponuje nativní funkcí inpolygon, byl v Pythonu ořez mapové kresby vyřešen efektivním využitím třídy Path z modulu matplotlib.path. Křivka polygonu definuje hranici a funkce hromadně testuje, zda do ní body promítnutého kontinentu spadají:

```
def clipLine(x, y, polyX, polyY):
    # Create a bounding polygon using matplotlib.path.Path
    poly = Path(np.column_stack((polyX, polyY)))
    inside = np.zeros(len(x), dtype=bool)

    # Filter out NaN values before testing for inclusion
    valid = ~np.isnan(x) & ~np.isnan(y)
    if np.any(valid):
        pts = np.column_stack((x[valid], y[valid]))
        # Bulk test whether points fall strictly inside the polygon
        inside[valid] = poly.contains_points(pts)

    x = x.copy()
    y = y.copy()
    # Assign NaN to points outside the boundary to break the line
    x[~inside] = np.nan
    y[~inside] = np.nan
    return x, y
```

2. Barycentrické mapování do 3D prostoru (map2DTo3D) Zpětné namapování oříznutých 2D linií na 3D stěnu polyedru zůstává matematicky shodné s předchozím řešením. Bod je rozložen na váhy l_1, l_2 a l_3, které jsou následně aplikovány přímo na prostorové souřadnice 3D trojúhelníku (s ohledem na indexování Pythonu od nuly):

```
def map2DTo3D(x, y, polyX, polyY, tri3D):
    # Extract vertices of the 2D projection polygon
    x1, y1 = polyX[0], polyY[0]
    x2, y2 = polyX[1], polyY[1]
    x3, y3 = polyX[2], polyY[2]

    # Denominator for barycentric coordinate computation
    den = (y2 - y3) * (x1 - x3) + (x3 - x2) * (y1 - y3)

    # Calculate barycentric weights (l1, l2)
    l1 = ((y2 - y3) * (x - x3) + (x3 - x2) * (y - y3)) / den
    l2 = ((y3 - y1) * (x - x3) + (x1 - x3) * (y - y3)) / den
    # The third weight ensures the sum is exactly 1.0
    l3 = 1 - l1 - l2

    # Apply weights to the 3D triangle vertices to get spatial coordinates
    return l1 * tri3D[0] + l2 * tri3D[1] + l3 * tri3D[2]
```

3. Vykreslení stěny a ošetření Z-fightingu (drawFace) Hlavní logika pro vykreslení prochází jednotlivé kontinenty, ošetřuje singularitu (sMin = np.deg2rad(5)) a pro zamezení prolínání mapy se samotnou plochou stěny (z-fighting) vypočítá plošnou normálu. Mapové linie jsou následně podél této normály mírně vysunuty (offset = 0.01):

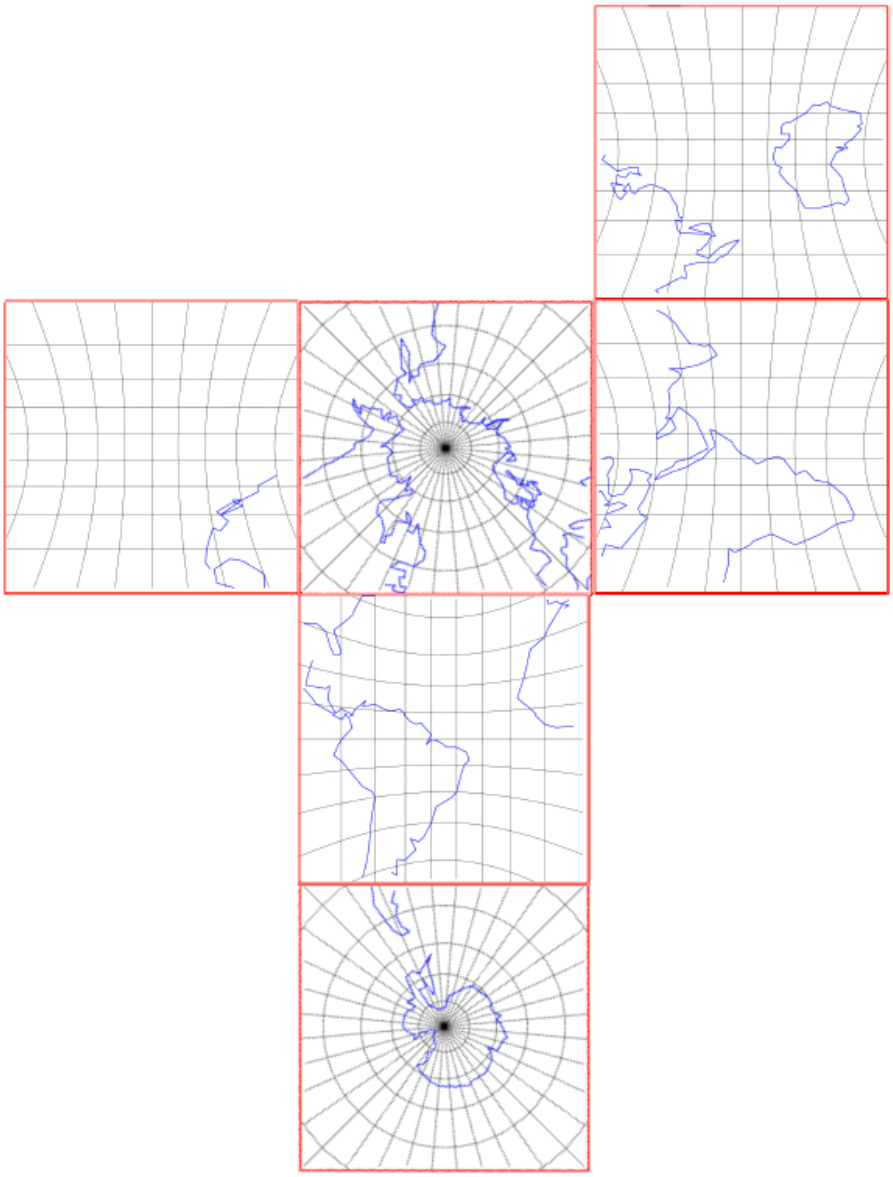
```
def drawFace(ax, face, V, conts):
    R = 1
    sMin = np.deg2rad(5)
    offset = 0.01  # Small offset to avoid z-fighting

    # Get boundary polygon coordinates
    polyX, polyY = boundary(
        R,
        face["uk"], face["vk"],
        face["ub"], face["vb"]
    )

    tri3D = V[np.array(face["idx"]) - 1]

    # Compute the outward normal vector of the 3D face
    n = np.cross(tri3D[1] - tri3D[0], tri3D[2] - tri3D[0])
    n = n / np.linalg.norm(n)

    # Ensure the normal points outwards from the body center
    if np.dot(n, tri3D.mean(axis=0)) < 0:
        n = -n

    # ... (iteration over continents and clipping) ...

            if len(xs) >= 2:
                # Map 2D segment coordinates back onto the 3D surface
                mapped_pts = []
                for i in range(len(xs)):
                    pt3D = map2DTo3D(
                        xs[i], ys[i],
                        polyX, polyY,
                        tri3D
                    )
                    mapped_pts.append(pt3D)

                P3 = np.array(mapped_pts)

                # Push the line slightly above the surface
                # along the normal vector
                P3 = P3 + (offset * n)

                # Plot the 3D polyline
                ax.plot(
                    P3[:, 0], P3[:, 1], P3[:, 2],
                    color="black",
                    linewidth=1.0
                )
```

5. Obrazové přílohy a realizace modelů

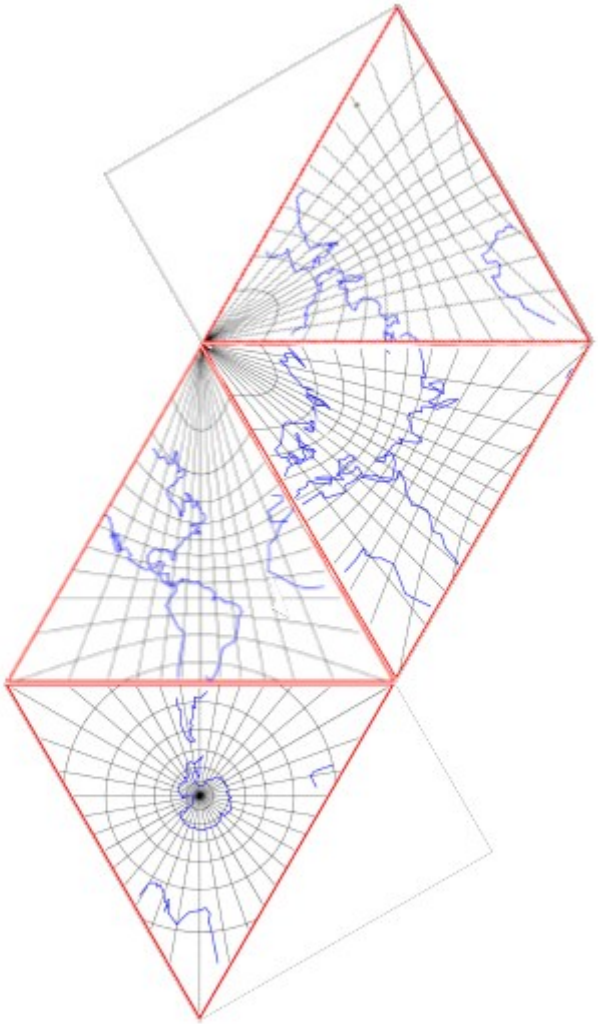
Tato kapitola obsahuje grafické výstupy vygenerované výpočetními skripty a dokumentuje fyzickou realizaci polyedrických globů. Všechny 2D modely byly vyexportovány z prostředí MATLAB a připraveny pro tisk v měřítku odpovídajícím zadání.

5.1 Rozložené sítě polyedrických globů

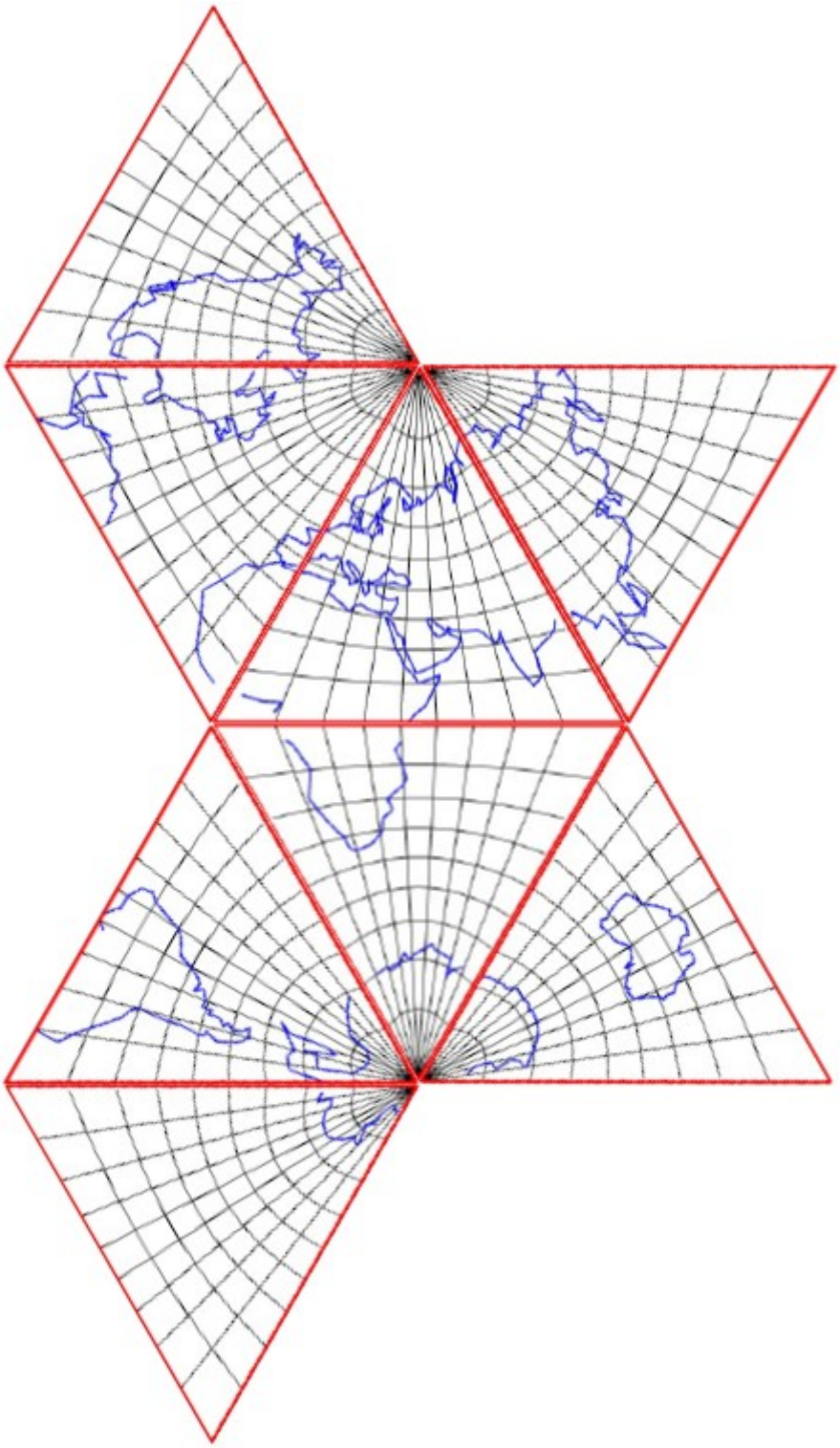
Následující obrazové přílohy představují kompletní rozložené sítě pro jednotlivá platonská tělesa. Sítě obsahují transformovanou geografickou mřížku a obrysy kontinentů zobrazené v gnomonické projekci. Topologie stěn byla zvolena tak, aby po vytištění a složení podél hran vznikl souvislý 3D model s navazující kresbou.



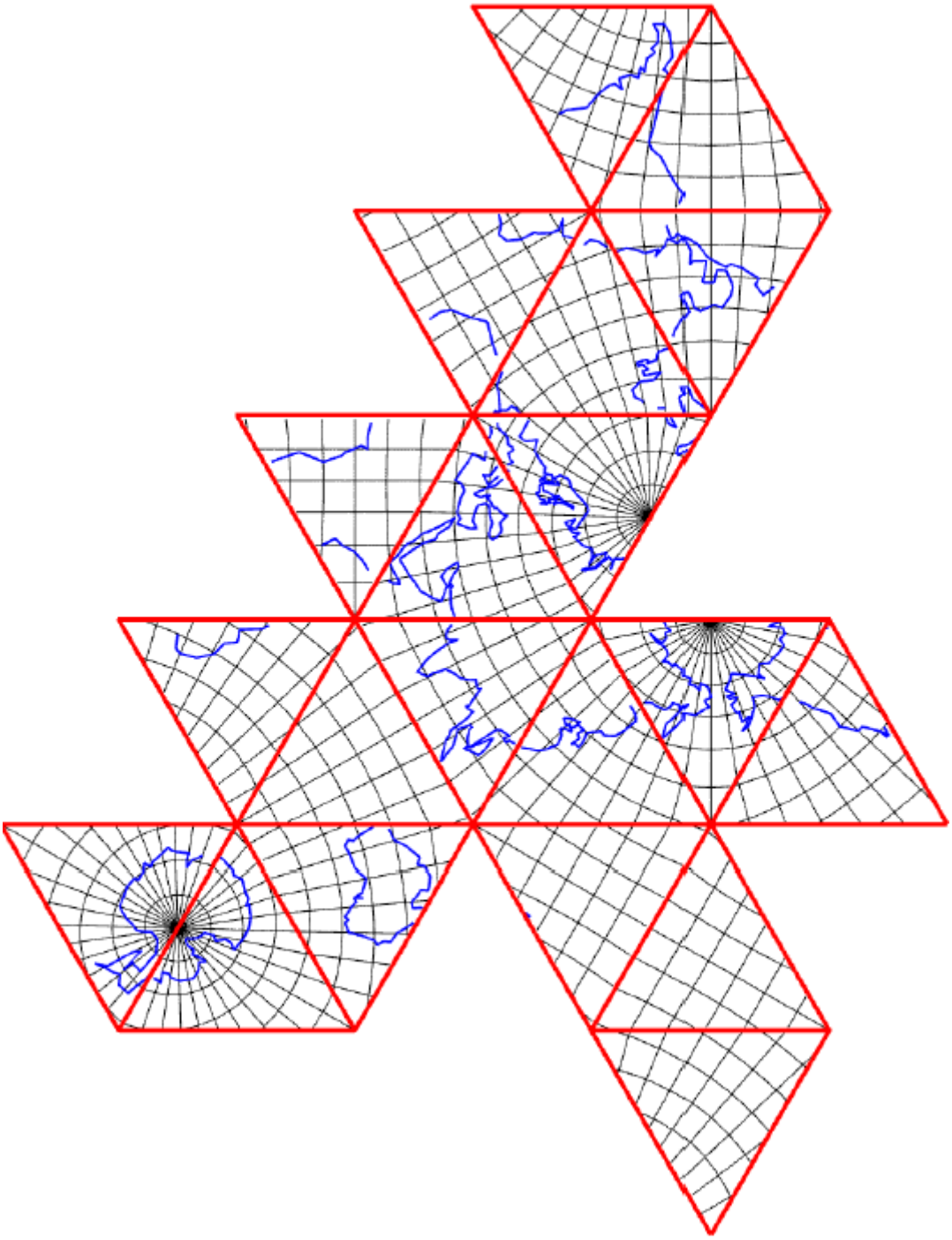
Obr. 1: Rozložená síť pro pravidelný šestistěn (krychli).



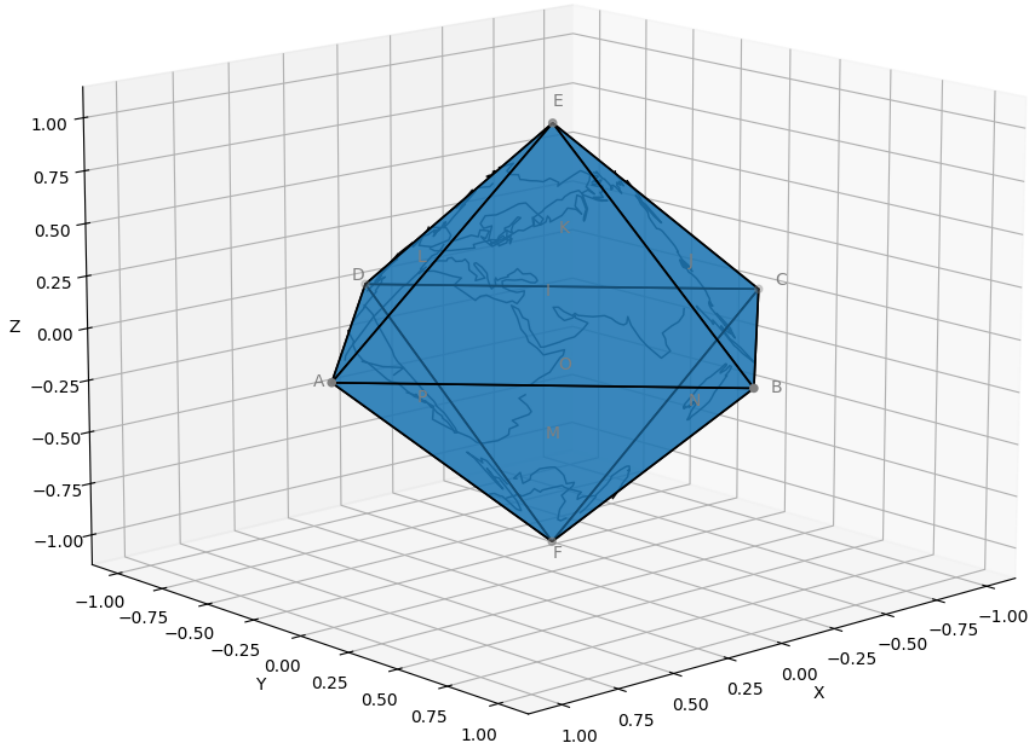
Obr. 2: Rozložená síť pro pravidelný čtyřstěn.



Obr. 3: Rozložená síť pro pravidelný osmistěn.



Obr. 4: Rozložená síť pro pravidelný dvacetistěn (ikozaedr).



Obr. 5: 3D model osmistěnu s mapou na povrchu

6. Tabulky výsledků

Platonské těleso	*Počet stěn*	*Tvar stěny*	*Zeměpisná šířka pólu (uk)*	*Max. sférická vzdálenost (z)*
Čtyřstěn	4	Trojúhelník	19.4712°	70° 31' 44"
Šestistěn	6	Čtverec	35.2644°	54° 44' 08"
Osmistěn	8	Trojúhelník	35.2644°	54° 44' 08"
Dvacetistěn	20	Trojúhelník	26.5651°	37° 22' 39"

Tabulka 1: Geometrické parametry stěn platonských těles.

Parametr zkreslení	*Vzorec*	*Čtyřstěn*	*Šestistěn / Osmistěn*	*Dvacetistěn*
Max. poloosa indikatrix (a)	1 / sin^2(s)	9.000000	3.000000	1.583592
Min. poloosa indikatrix (b)	1 / sin(s)	3.000000	1.732051	1.258409
Plošné měřítko (P)	a * b	27.000000	5.196152	1.992806
Max. úhlové zkreslení (Δω)	2 arcsin((a-b)/(a+b))	60° 00' 00"	31° 05' 04"	13° 08' 11"

Tabulka 2: Extrémní charakteristiky zkreslení v okrajovém bodě Q

Směr zkreslení	*Skutečná délka na sféře (ds)*	*Délka obrazu v PROJ (dp)*	*Vypočtené lokální měřítko (m)*
V poledníku (mp)	11.135201 m	17.084074 m	1.534658
V rovnoběžce (mr)	10.966445 m	16.975541 m	1.547975

Tabulka 3: Nezávislá validace lokálního zkreslení v testovacím bodě Q pomocí knihovny PROJ

Těleso	*Souřadnice pólu stěny (K)*	*Testovací bod (Q)*	*Meridiánová konvergence (γ)*
Čtyřstěn	[19.4712°, 60°]	[−0.2°, 20°]	12.190135°
Šestistěn	[0°, 45°]	[−0.7°, 20°]	4.881261°
Osmistěn	[35.2644°, 45°]	[−0.7°, 20°]	22.355104°

Tabulka 4: Ukázkové hodnoty meridiánové konvergence získané numerickou derivací z vlastních skriptů.

7. Zhodnocení a závěr

Předložená práce se zabývala problematikou konstrukce polyedrických globů s využitím gnomonické projekce. V rámci projektu byly úspěšně algoritmizovány výpočty pro čtyři základní platonská tělesa: pravidelný čtyřstěn, šestistěn, osmistěn a bonusový dvacetistěn.

7.1 Shrnutí kartografických poznatků

Z analýzy zkreslení vyplývá přímá úměra mezi počtem stěn tělesa a kvalitou kartografického zobrazení. Zatímco u čtyřstěnu dosahuje plošné zkreslení v rozích stěn extrémních hodnot ($P = 27$) a úhlové zkreslení činí $60°$, u dvacetistěnu klesá plošné zkreslení na hodnotu $P \approx 1{,}99$ a úhlová deformace na přijatelných $13°$. Tento nelineární pokles zkreslení potvrzuje, že ikozaedr je z hlediska matematické kartografie nejvhodnější aproximací sféry pro účely polyedrických modelů.

Gnomonická projekce se pro tento účel ukázala jako ideální, neboť promítá velké kružnice (včetně hran těles) jako úsečky, což usnadňuje fyzickou kompletaci modelu. Hlavní nevýhoda zobrazení — extrémní zkreslení s rostoucí vzdáleností od pólu — byla efektivně potlačena rozdělením sféry na více dílčích projekčních rovin.

7.2 Technické zhodnocení a bonusové úlohy

Programová realizace v prostředí MATLAB prokázala vysokou efektivitu díky modulární struktuře skriptů. Podařilo se vyřešit několik netriviálních technických problémů:

Topologická integrita: Implementace dynamického prahu (jumpThreshold) zajistila správné vykreslení kontinentů bez falešných spojnic přes projekční horizont.

Numerická derivace: Výpočet meridiánové konvergence pomocí diferencí umožnil získat přesné hodnoty stočení poledníků bez nutnosti složitého analytického vyjadřování parciálních derivací.

Analytická konstrukce: Dvacetistěn byl generován čistě na základě geometrických principů (zlatý řez), což eliminovalo potřebu manuálního zadávání souřadnic.

Nezávislá validace: Shoda výsledků s knihovnou PROJ potvrdila matematickou správnost vlastních algoritmů.

7.3 Závěr

Cíle cvičení byly naplněny v celém rozsahu. Výsledné PDF exporty a sestavené fyzické modely vykazují vysokou míru geometrické přesnosti a vizuální spojitosti. Projekt demonstruje, že i s využitím klasických projekčních rovnic lze dosáhnout moderních výsledků v oblasti tvorby glóbů a prostorových vizualizací Země.

8. Seznam literatury a použitých zdrojů

1. **BAYER, Tomáš.** *Matematická kartografie* [online]. Přírodovědecká fakulta Univerzity Karlovy [cit. 2026-04-24]. Dostupné z: https://web.natur.cuni.cz/~bayertom/index.php/teaching/matematicka-kartografie
2. **OPENAI.** *ChatGPT* [Velký jazykový model]. 2026. Dostupné z: https://chatgpt.com. (Využito pro konzultace při tvorbě 3D algoritmů a optimalizaci skriptů pro dvacetistěn).